

CMS/CS/EE/IDS 144 HW 5: Centrality & Games

Maurice Hanisch

February 17, 2026

1 Discovering Clusters

1.1 Symmetric Stochastic Block Model (SSBM)

In this part, we generated a SSBM network with $n = 30$, $k = 3$, $A = 0.7$, and $B = 0.1$. We compared k -means and spectral clustering algorithms using the implementation in Listing 2.

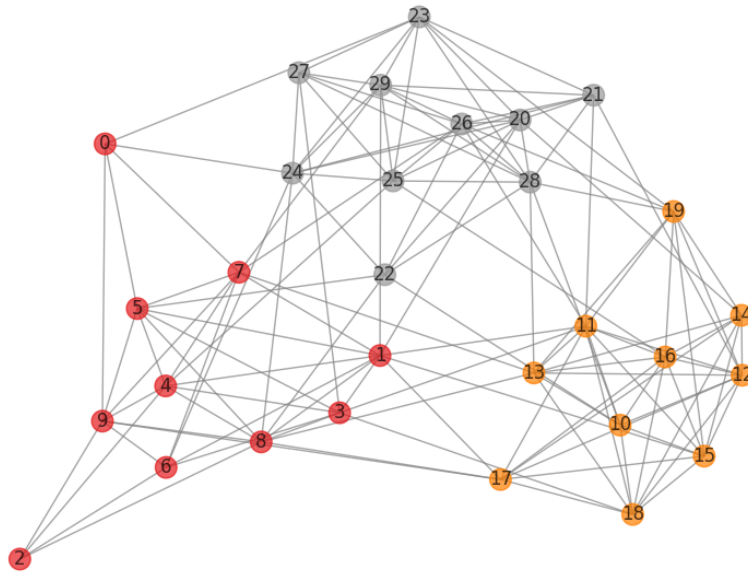


Figure 1: Ground Truth Clusters for SSBM

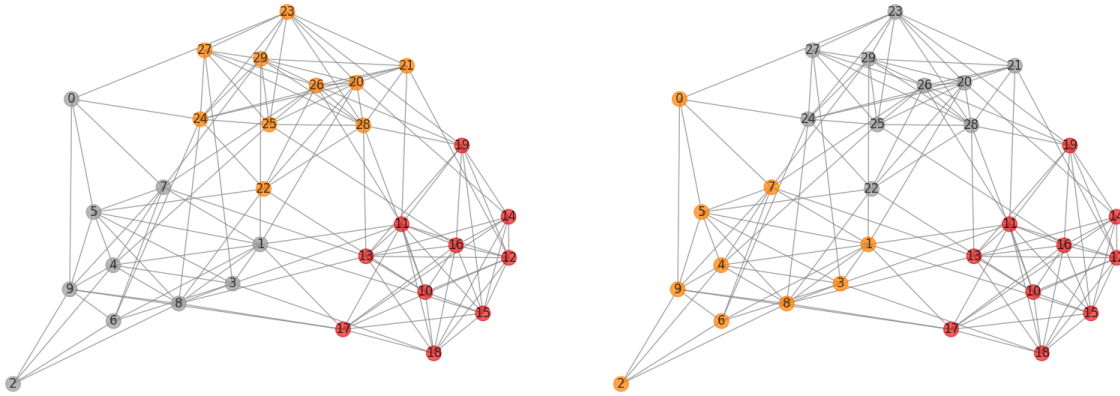


Figure 2: Clustering Results for SSBM: K-Means (left) and Spectral Clustering (right)

Discussion: Both K-Means and Spectral Clustering recovered the ground truth clusters (Figure 1) perfectly, as shown in Figure 2 ($ARI = 1.000$).

As highlighted in Note 1 of the assignment, the two algorithms interpret the data differently. For the SSBM graph, which is natively represented by its adjacency matrix:

- **K-Means Interpretation:** We pass the adjacency matrix as input. K-Means interprets each row (or column) as a point in n -dimensional space. Each row acts as a "connectivity signature" or "neighborhood feature vector." Since nodes within the same community have a much higher probability of connecting to each other ($A = 0.7$) than to others ($B = 0.1$), their signatures will be geometrically close in $\mathbb{R}^n$. The Euclidean distance between rows effectively measures the similarity of their connection patterns, allowing K-Means to find the clusters.
- **Spectral Clustering Affinity:** Since we are already working with an adjacency matrix that defines the graph structure, we use `affinity='precomputed'`. This choice treats the adjacency matrix itself as the similarity matrix. This follows the conventional definition of spectral clustering on graphs, where the algorithm leverages the spectrum of the Laplacian derived from the connectivity to embed the nodes in a low-dimensional space before clustering.

The perfect recovery demonstrates that for this level of noise ($B = 0.1$ vs $A = 0.7$), both the direct neighborhood signatures and the spectral properties of the graph are highly discriminative.

1.2 Circle Graph

We generated a circle graph with $n = 500$ and $\sigma = 0.01$ using the code in Listing 3.

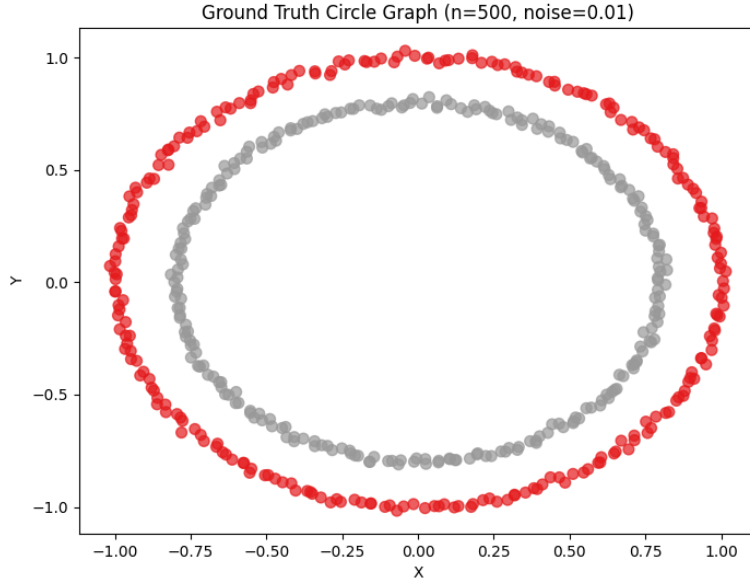


Figure 3: Ground Truth Clusters for Circle Graph

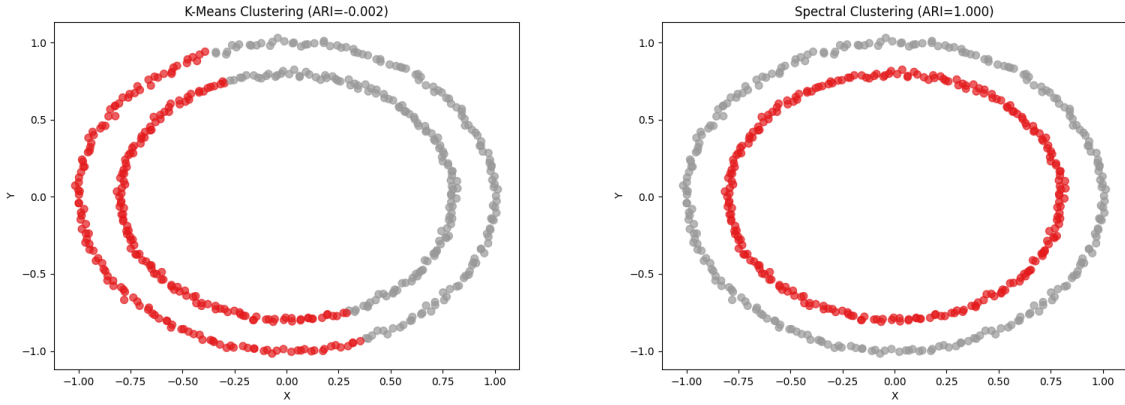


Figure 4: Clustering Results for Circle Graph: K-Means (left) and Spectral Clustering (right)

Discussion: There is a stark difference between the two algorithms here (see Figure 4). K-Means ($ARI \approx 0$) fails completely because it assumes clusters are convex and separable by linear boundaries. Since one circle is nested inside the other (Figure 3), no linear hyperplane can separate them. Spectral Clustering ($ARI = 1.000$) handles this perfectly. Using the `nearest_neighbors` affinity function (as implemented in Listing 3), it constructs a graph where edges only exist between nearby points. The inner and outer circles form two separate connected components, which the spectral embedding maps into highly separable regions in the eigen-space.

2 Braess's Paradox Simulation

2.1 Initial Traffic Network (Part a)

We simulated 4000 drivers on the $S \rightarrow A \rightarrow E$ and $S \rightarrow B \rightarrow E$ paths using the simulation engine in Listing 4 and the runner in Listing 5.

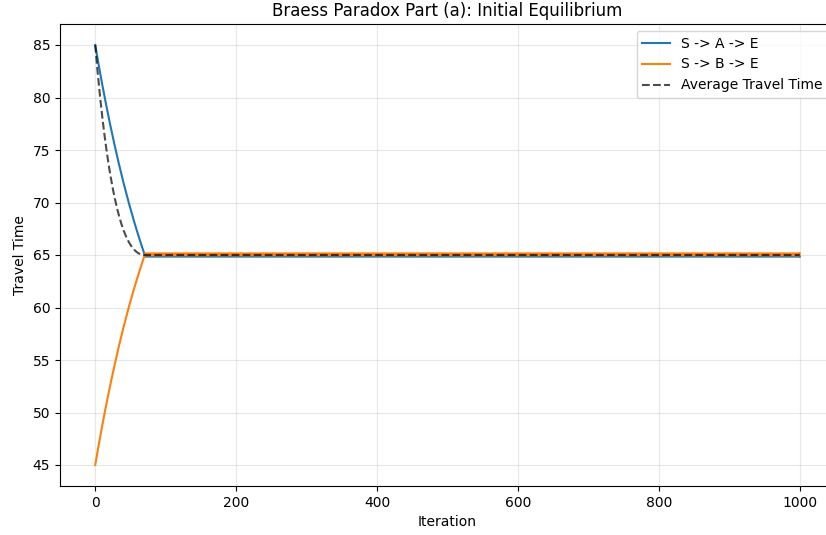


Figure 5: Traffic equilibrium without the shortcut.

Results: As shown in Figure 5, the system reached equilibrium with approximately 2000 drivers on each path (Final distribution: Top = 2004, Bottom = 1996). The final travel times were: Path 1 ≈ 65.04 , Path 2 ≈ 64.96 . The average travel time was 65.00. This is an equilibrium because the costs are nearly identical, and any driver switching paths would not significantly improve their time.

2.2 Adding the Shortcut (Part b)

We added the $A \rightarrow B$ edge with $T(x) = 0$.

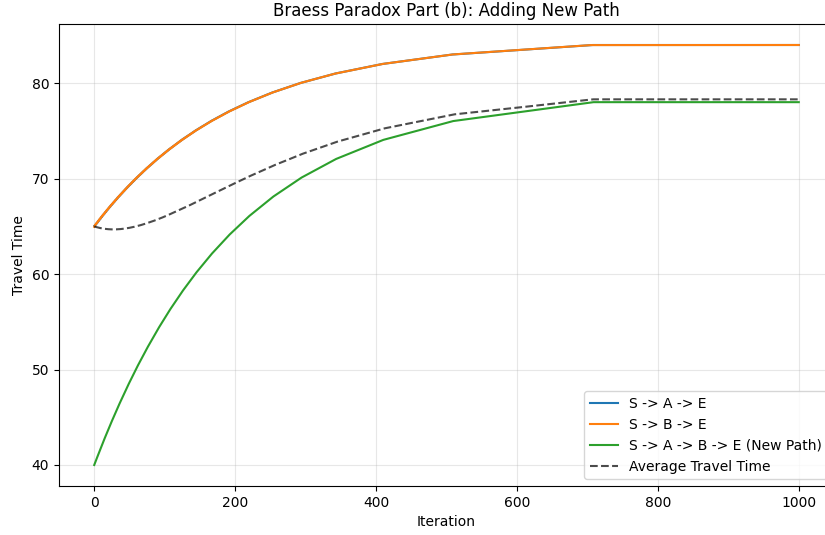


Figure 6: Simulation of Braess's Paradox after adding $A \rightarrow B$.

Results: The simulation results in Figure 6 show that the new distribution moved nearly all drivers to the middle path $S \rightarrow A \rightarrow B \rightarrow E$ (Final distribution: Top = 99, Bottom = 99, Middle = 3802). The new average travel time is approximately 78.32. Compared to the initial 65.00 in Figure 5, the travel time **increased** significantly. This confirms Braess's Paradox: adding capacity (a free shortcut) worsened the global outcome.

This is still an equilibrium. At the final state:

- Middle path cost: $(3802 + 99)/100 + 0 + (3802 + 99)/100 = 78.02$.
- Top path cost: $(3802 + 99)/100 + 45 = 84.01$.
- Bottom path cost: $45 + (3802 + 99)/100 = 84.01$.

Individual drivers have no incentive to switch back to the original paths because $78.02 < 84.01$.

2.3 Intuitive Explanation (Part c)

Drivers flock to the new path because, from an individual's perspective, the $S \rightarrow A \rightarrow B \rightarrow E$ path is always better or equal to the alternatives for a given distribution of others. Specifically, for any flow x_{SA} and x_{BE} , the path $S \rightarrow A \rightarrow B \rightarrow E$ has cost $x_{SA}/100 + x_{BE}/100$, while $S \rightarrow A \rightarrow E$ has $x_{SA}/100 + 45$. Since $x_{BE}/100$ can't exceed 40, the middle path looks cheaper than the top one. However, when everyone chooses this "shortcut", the congestion on the shared segments ($S \rightarrow A$ and $B \rightarrow E$) increases so much that the total time exceeds the original state. They don't switch back because even in the congested state, the shortcut is still faster than the alternatives *given the current congestion*. This is a classic example of a Nash Equilibrium that is not socially optimal.

3 Routing Games with Tolls

Problem 5: Prove that any optimal strategy for our original game is a Nash equilibrium for this new game (with link cost functions $c_e^*(\cdot)$).

Solution:

In an atomic routing game where each user has traffic $r_i = 1$, we can analyze the behavior of selfish players by constructing a potential function. A game with edge cost functions $\tilde{c}_e(x)$ is an exact potential game with Rosenthal's potential function:

$$\Phi(f) = \sum_{e \in E} \sum_{k=1}^{f_e} \tilde{c}_e(k)$$

where f_e is the number of players using edge e . By the properties of exact potential games, any flow profile f that minimizes the potential function $\Phi(f)$ is a pure-strategy Nash Equilibrium.

For the new game, the cost functions are given by marginal cost pricing:

$$c_e^*(x) = c_e(x) + (x-1)(c_e(x) - c_e(x-1))$$

We can rewrite the expression for $c_e^*(x)$ as follows:

$$c_e^*(x) = xc_e(x) - (x-1)c_e(x-1)$$

Now, let's construct the potential function $\Phi_{new}(f)$ for this new game by substituting $c_e^*(x)$ into Rosenthal's potential function formula:

$$\begin{aligned} \Phi_{new}(f) &= \sum_{e \in E} \sum_{k=1}^{f_e} c_e^*(k) \\ \Phi_{new}(f) &= \sum_{e \in E} \sum_{k=1}^{f_e} [kc_e(k) - (k-1)c_e(k-1)] \end{aligned}$$

Observe that the inner sum over k is a telescoping sum. Let $S_k = kc_e(k)$. The term inside the sum is exactly $S_k - S_{k-1}$. Expanding the sum for a given edge e , we get:

$$\sum_{k=1}^{f_e} [kc_e(k) - (k-1)c_e(k-1)] = (1 \cdot c_e(1) - 0) + (2 \cdot c_e(2) - 1 \cdot c_e(1)) + \dots + (f_e c_e(f_e) - (f_e - 1)c_e(f_e - 1))$$

All intermediate terms cancel out, leaving only the final term:

$$\sum_{k=1}^{f_e} c_e^*(k) = f_e c_e(f_e)$$

Substituting this back into the potential function gives:

$$\Phi_{new}(f) = \sum_{e \in E} f_e c_e(f_e)$$

Notice that $\Phi_{new}(f)$ is exactly the objective function (social cost) $C(f)$ of the *original* game!

$$\Phi_{new}(f) = C(f) = \sum_{e \in E} f_e c_e(f_e)$$

Let f^* be an optimal strategy (flow profile) for the original game. By definition, f^* minimizes the social cost $C(f)$. Therefore, f^* must also minimize the potential function $\Phi_{new}(f)$ of the new game.

Since f^* is a global minimum of the exact potential function $\Phi_{new}(f)$, no player can unilaterally deviate to lower their personal cost (as any such deviation would strictly decrease the potential, which is impossible at a global minimum). Thus, f^* is a Nash equilibrium for the new game.

A Code Listings

A.1 Clustering Utilities

The utilities for generating SSBM and visualizing graphs are provided in Listing 1.

```
1 import numpy as np
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 from sklearn.metrics import adjusted_rand_score
5
6 def generate_ssbm(n, k, A, B):
7     """
8     Generate a Symmetric Stochastic Block Model (SSBM) graph.
9     n: Total number of nodes
10    k: Number of communities
11    A: Probability of edge within community
12    B: Probability of edge between communities
13    """
14    labels = np.repeat(np.arange(k), n // k)
15    # If n is not divisible by k, distribute the remaining nodes
16    remainder = n % k
17    if remainder > 0:
18        labels = np.concatenate([labels, np.arange(remainder)])
19
20    adj = np.zeros((n, n))
21    for i in range(n):
22        for j in range(i + 1, n):
23            prob = A if labels[i] == labels[j] else B
24            if np.random.rand() < prob:
25                adj[i, j] = 1
26                adj[j, i] = 1
27
28    return adj, labels
29
30 def plot_graph_clusters(adj, labels, title, save_path):
31    G = nx.from_numpy_array(adj)
32    pos = nx.spring_layout(G, seed=42)
33    plt.figure(figsize=(8, 6))
34    nx.draw(G, pos, node_color=labels, with_labels=True, cmap=plt.cm.Set1,
35            node_size=200, edge_color='gray', alpha=0.7)
36    plt.title(title)
37    plt.savefig(save_path)
38    plt.close()
39
40 def plot_scatter_clusters(X, labels, title, save_path):
41    plt.figure(figsize=(8, 6))
42    plt.scatter(X[:, 0], X[:, 1], c=labels, cmap=plt.cm.Set1, s=50, alpha=0.7)
43    plt.title(title)
44    plt.xlabel("X")
45    plt.ylabel("Y")
46    plt.savefig(save_path)
47    plt.close()
48
49 def calculate_metrics(true_labels, pred_labels):
50    ari = adjusted_rand_score(true_labels, pred_labels)
51    # Simple accuracy (considering label permutations)
52    # For small k, we can find the best permutation, but ARI is generally better
53    for clustering
```

```
52     return ari
```

Listing 1: clustering_utils.py

A.2 Problem 1a Solver

The experimental setup for SSBM clustering is shown in Listing 2.

```
1 import os
2 import numpy as np
3 from sklearn.cluster import KMeans, SpectralClustering
4 from clustering_utils import generate_ssbm, plot_graph_clusters, calculate_metrics
5
6 def solve_p1a():
7     # Parameters
8     n = 30
9     k = 3
10    A = 0.7
11    B = 0.1
12
13    # Ensure figs directory exists
14    os.makedirs('latex/figs', exist_ok=True)
15
16    # Generate SSBM
17    adj, true_labels = generate_ssbm(n, k, A, B)
18
19    # 1. K-Means
20    # Note: K-means on adjacency matrix treats each row (neighborhood vector) as a
21    # point in  $\mathbb{R}^n$ 
22    kmeans = KMeans(n_clusters=k, random_state=42)
23    kmeans_labels = kmeans.fit_predict(adj)
24
25    # 2. Spectral Clustering
26    # For adjacency matrix, we use affinity='precomputed' or 'nearest_neighbors'
27    # Actually, SpectralClustering defaults to 'rbf'. For a graph, we often use '
28    # nearest_neighbors' or 'precomputed'
29    spectral = SpectralClustering(n_clusters=k, affinity='precomputed',
30    random_state=42)
31    spectral_labels = spectral.fit_predict(adj)
32
33    # Plotting
34    plot_graph_clusters(adj, true_labels, f"Ground Truth SSBM (n={n}, k={k}, A={A
35    }, B={B})", "latex/figs/p1a_ground_truth.png")
36    plot_graph_clusters(adj, kmeans_labels, f"K-Means Clustering (ARI={
37    calculate_metrics(true_labels, kmeans_labels):.3f})", "latex/figs/p1a_kmeans.
38    png")
39    plot_graph_clusters(adj, spectral_labels, f"Spectral Clustering (ARI={
40    calculate_metrics(true_labels, spectral_labels):.3f})", "latex/figs/
41    p1a_spectral.png")
42
43    print(f"P1a: K-Means ARI: {calculate_metrics(true_labels, kmeans_labels):.3f}"
44    )
45    print(f"P1a: Spectral ARI: {calculate_metrics(true_labels, spectral_labels):.3
46    f}")
47
48 if __name__ == "__main__":
49     solve_p1a()
```

Listing 2: solve_p1a.py

A.3 Problem 1b Solver

The experimental setup for Circle graph clustering is shown in Listing 3.

```
1 import os
2 import numpy as np
3 from sklearn.datasets import make_circles
4 from sklearn.cluster import KMeans, SpectralClustering
5 from clustering_utils import plot_scatter_clusters, calculate_metrics
6
7 def solve_p1b():
8     # Parameters
9     n = 500
10    noise = 0.01
11    factor = 0.8 # Default as per Note 2
12    k = 2
13
14    # Ensure figs directory exists
15    os.makedirs('latex/figs', exist_ok=True)
16
17    # Generate Circle Graph
18    X, true_labels = make_circles(n_samples=n, noise=noise, factor=factor,
19    random_state=42)
20
21    # 1. K-Means
22    kmeans = KMeans(n_clusters=k, random_state=42)
23    kmeans_labels = kmeans.fit_predict(X)
24
25    # 2. Spectral Clustering
26    # For circle graph, we use affinity='nearest_neighbors' as suggested in Note 1
27    spectral = SpectralClustering(n_clusters=k, affinity='nearest_neighbors',
28    n_neighbors=10, random_state=42)
29    spectral_labels = spectral.fit_predict(X)
30
31    # Plotting
32    plot_scatter_clusters(X, true_labels, f"Ground Truth Circle Graph (n={n},
33    noise={noise})", "latex/figs/p1b_ground_truth.png")
34    plot_scatter_clusters(X, kmeans_labels, f"K-Means Clustering (ARI={
35    calculate_metrics(true_labels, kmeans_labels):.3f})", "latex/figs/p1b_kmeans.
36    png")
37    plot_scatter_clusters(X, spectral_labels, f"Spectral Clustering (ARI={
38    calculate_metrics(true_labels, spectral_labels):.3f})", "latex/figs/
39    p1b_spectral.png")
40
41    print(f"P1b: K-Means ARI: {calculate_metrics(true_labels, kmeans_labels):.3f}"
42    )
43    print(f"P1b: Spectral ARI: {calculate_metrics(true_labels, spectral_labels):.3
44    f}")
45
46 if __name__ == "__main__":
47     solve_p1b()
```

Listing 3: solve_p1b.py

A.4 Braess Simulation Engine

The core simulation logic for Braess's Paradox is implemented in Listing 4.

```
1 import numpy as np
```

```

2 import matplotlib.pyplot as plt
3
4 class BraessSimulator:
5     def __init__(self, num_drivers=4000):
6         self.num_drivers = num_drivers
7
8     def get_costs_part_a(self, n_top, n_bottom):
9         """
10         Path 1: S -> A -> E:  $T(x) = x/100 + 45$ 
11         Path 2: S -> B -> E:  $T(x) = 45 + x/100$ 
12         n_top: drivers on Path 1
13         n_bottom: drivers on Path 2
14         """
15         cost_top = n_top / 100 + 45
16         cost_bottom = 45 + n_bottom / 100
17         return cost_top, cost_bottom
18
19     def get_costs_part_b(self, n_top, n_bottom, n_middle):
20         """
21         Path 1: S -> A -> E:  $T(x) = x_{SA}/100 + T_{AE}(x_{AE})$ 
22         Path 2: S -> B -> E:  $T(x) = T_{SB}(x_{SB}) + x_{BE}/100$ 
23         Path 3: S -> A -> B -> E:  $T(x) = x_{SA}/100 + 0 + x_{BE}/100$ 
24
25         Flows:
26         x_SA = n_top + n_middle
27         x_AE = n_top
28         x_SB = n_bottom
29         x_BE = n_bottom + n_middle
30
31         Costs:
32         T_SA = x_SA / 100
33         T_AE = 45
34         T_SB = 45
35         T_BE = x_BE / 100
36         T_AB = 0
37         """
38         x_SA = n_top + n_middle
39         x_BE = n_bottom + n_middle
40
41         cost_top = x_SA / 100 + 45
42         cost_bottom = 45 + x_BE / 100
43         cost_middle = x_SA / 100 + 0 + x_BE / 100
44
45         return cost_top, cost_bottom, cost_middle
46
47     def run_simulation(self, part='a', iterations=1000, initial_dist=None):
48         if part == 'a':
49             if initial_dist is None:
50                 n_top, n_bottom = self.num_drivers, 0
51             else:
52                 n_top, n_bottom = initial_dist
53
54             history = {'top': [], 'bottom': [], 'average': []}
55
56             for _ in range(iterations):
57                 c_top, c_bottom = self.get_costs_part_a(n_top, n_bottom)
58                 history['top'].append(c_top)
59                 history['bottom'].append(c_bottom)

```

```

60         avg_time = (n_top * c_top + n_bottom * c_bottom) / self.
num_drivers
61         history['average'].append(avg_time)
62
63         # Move 1% from slowest to fastest
64         move_count = int(0.01 * (n_top if c_top > c_bottom else n_bottom))
65         if c_top > c_bottom:
66             n_top -= move_count
67             n_bottom += move_count
68         elif c_bottom > c_top:
69             n_bottom -= move_count
70             n_top += move_count
71
72         return (n_top, n_bottom), history
73
74     elif part == 'b':
75         # initial_dist from part a: (n_top, n_bottom)
76         n_top, n_bottom = initial_dist
77         n_middle = 0
78
79         history = {'top': [], 'bottom': [], 'middle': [], 'average': []}
80
81         for _ in range(iterations):
82             c_top, c_bottom, c_middle = self.get_costs_part_b(n_top, n_bottom,
n_middle)
83             costs = [c_top, c_bottom, c_middle]
84             history['top'].append(c_top)
85             history['bottom'].append(c_bottom)
86             history['middle'].append(c_middle)
87             avg_time = (n_top * c_top + n_bottom * c_bottom + n_middle *
c_middle) / self.num_drivers
88             history['average'].append(avg_time)
89
90             # Move 1% from slowest path to fastest path
91             # Identify slowest and fastest
92             slowest_idx = np.argmax(costs)
93             fastest_idx = np.argmin(costs)
94
95             if slowest_idx == fastest_idx:
96                 continue
97
98             drivers = [n_top, n_bottom, n_middle]
99             move_count = int(0.01 * drivers[slowest_idx])
100
101             drivers[slowest_idx] -= move_count
102             drivers[fastest_idx] += move_count
103
104             n_top, n_bottom, n_middle = drivers
105
106         return (n_top, n_bottom, n_middle), history

```

Listing 4: braess_simulation.py

A.5 Problem 2 Solver

The script used to run the traffic simulations is shown in Listing 5.

```

1 import os
2 import matplotlib.pyplot as plt

```

```

3 from braess_simulation import BraessSimulator
4
5 def solve_p2():
6     sim = BraessSimulator(num_drivers=4000)
7     os.makedirs('latex/figs', exist_ok=True)
8
9     # Part (a)
10    final_dist_a, history_a = sim.run_simulation(part='a', iterations=1000)
11
12    plt.figure(figsize=(10, 6))
13    plt.plot(history_a['top'], label='S -> A -> E')
14    plt.plot(history_a['bottom'], label='S -> B -> E')
15    plt.plot(history_a['average'], 'k--', label='Average Travel Time', alpha=0.7)
16    plt.xlabel('Iteration')
17    plt.ylabel('Travel Time')
18    plt.title('Braess Paradox Part (a): Initial Equilibrium')
19    plt.legend()
20    plt.grid(True, alpha=0.3)
21    plt.savefig('latex/figs/p2a_plot.png')
22    plt.close()
23
24    print(f"P2a Final Distribution: Top={final_dist_a[0]}, Bottom={final_dist_a[1]}")
25    c1, c2 = sim.get_costs_part_a(*final_dist_a)
26    print(f"P2a Final Costs: Path1={c1:.2f}, Path2={c2:.2f}, Avg={({final_dist_a[0]*c1 + final_dist_a[1]*c2})/4000:.2f}")
27
28    # Part (b)
29    final_dist_b, history_b = sim.run_simulation(part='b', iterations=1000,
30    initial_dist=final_dist_a)
31
32    plt.figure(figsize=(10, 6))
33    plt.plot(history_b['top'], label='S -> A -> E')
34    plt.plot(history_b['bottom'], label='S -> B -> E')
35    plt.plot(history_b['middle'], label='S -> A -> B -> E (New Path)')
36    plt.plot(history_b['average'], 'k--', label='Average Travel Time', alpha=0.7)
37    plt.xlabel('Iteration')
38    plt.ylabel('Travel Time')
39    plt.title('Braess Paradox Part (b): Adding New Path')
40    plt.legend()
41    plt.grid(True, alpha=0.3)
42    plt.savefig('latex/figs/p2b_plot.png')
43    plt.close()
44
45    print(f"P2b Final Distribution: Top={final_dist_b[0]}, Bottom={final_dist_b[1]}, Middle={final_dist_b[2]}")
46    c1, c2, c3 = sim.get_costs_part_b(*final_dist_b)
47    print(f"P2b Final Costs: Path1={c1:.2f}, Path2={c2:.2f}, Path3={c3:.2f}, Avg={({final_dist_b[0]*c1 + final_dist_b[1]*c2 + final_dist_b[2]*c3})/4000:.2f}")
48
49 if __name__ == "__main__":
50     solve_p2()

```

Listing 5: solve_p2.py